Booking Platform

Abstract:

The purpose of this project is to design and develop a web-based local service booking platform that addresses inefficiencies in traditional service discovery and scheduling. Many local service providers rely on manual processes or fragmented tools that lead to booking delays, scheduling conflicts, and limited pricing transparency. This project investigates how a centralized, bid-based booking system can improve efficiency, trust, and accessibility for both service seekers and providers.

The project follows an Agile development approach and focuses on implementing a digital marketplace where users can post service requests and receive competitive bids from local providers. Core system components include service request management, provider bidding, automated calendar updates to prevent double bookings, and a rating and review system to promote service quality.

The expected results of the project include reduced booking confirmation time, elimination of scheduling conflicts during testing, and improved user satisfaction through transparent pricing and verified feedback. Overall, the project concludes that an automated, competitive booking platform can significantly streamline local service interactions while creating a fair and efficient marketplace for users and service providers.

Introduction:

Today, many local service providers rely on manual scheduling methods, messaging applications, or fragmented platforms that often result in booking conflicts, delayed confirmations, and limited pricing transparency. Customers frequently experience difficulty finding available providers, comparing prices, and trusting service quality due to the lack of centralized information. The Booking Platform aims to address these challenges by offering a web-based system where users can post service requests, receive bids from providers, and confirm appointments through an organized and automated process.

Service seekers can submit requests based on preferred time, location, and service type, while providers can respond with their availability and pricing. Once a booking is confirmed, the system automatically updates the provider's calendar to prevent scheduling conflicts. Users are also able to leave ratings and reviews after service completion, helping future customers make informed decisions. This report focuses on the system's primary features and architecture, highlighting strengths such as automated scheduling, bid comparison, secure account management, and reliable data storage. Features that were excluded include online payment integration, third-party calendar synchronization, and mobile push notifications.

The main objective of the project is to develop a functional and user-friendly booking platform that simplifies service discovery and improves scheduling efficiency for both customers and providers. One of the challenges encountered during the design process was ensuring that bookings remain conflict-free while maintaining ease of use for different user roles. To address this, the team proposed an automated calendar-blocking mechanism and an Agile development approach that allowed continuous refinement of features throughout the project lifecycle.

Architecture: Development Perspective

Figure 1: Booking Platform High-level System Architecture Diagram

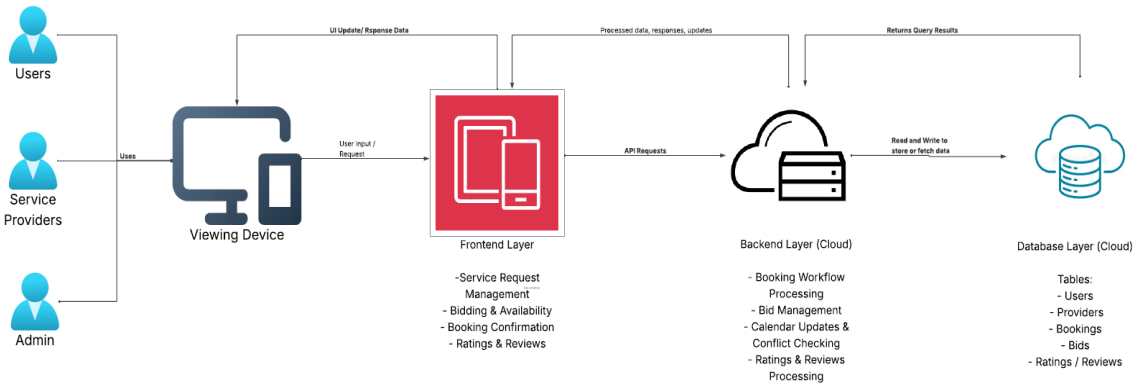

Figure 1: Booking Platform System Architecture Diagram

The booking platform consists of three main layers: the Viewing Device, the Frontend / User Interface, and the Cloud Backend with the business logic, which communicates with the Cloud Database as storage. Users, Providers, and Admin access the system through their devices, sending requests to the frontend and receiving interface updates in response. The frontend handles all interactions, including posting service requests, submitting bids, confirming bookings, rating providers, and viewing reports.

The cloud backend processes all business logic, such as booking workflows, bid management, calendar conflict checking, ratings and reviews, and reporting. It communicates directly with the cloud database to read and write persistent data, returning results to the frontend so the UI can update appropriately. This architecture provides a clear separation of concerns, centralized data management, and efficient communication between all hardware and software components.

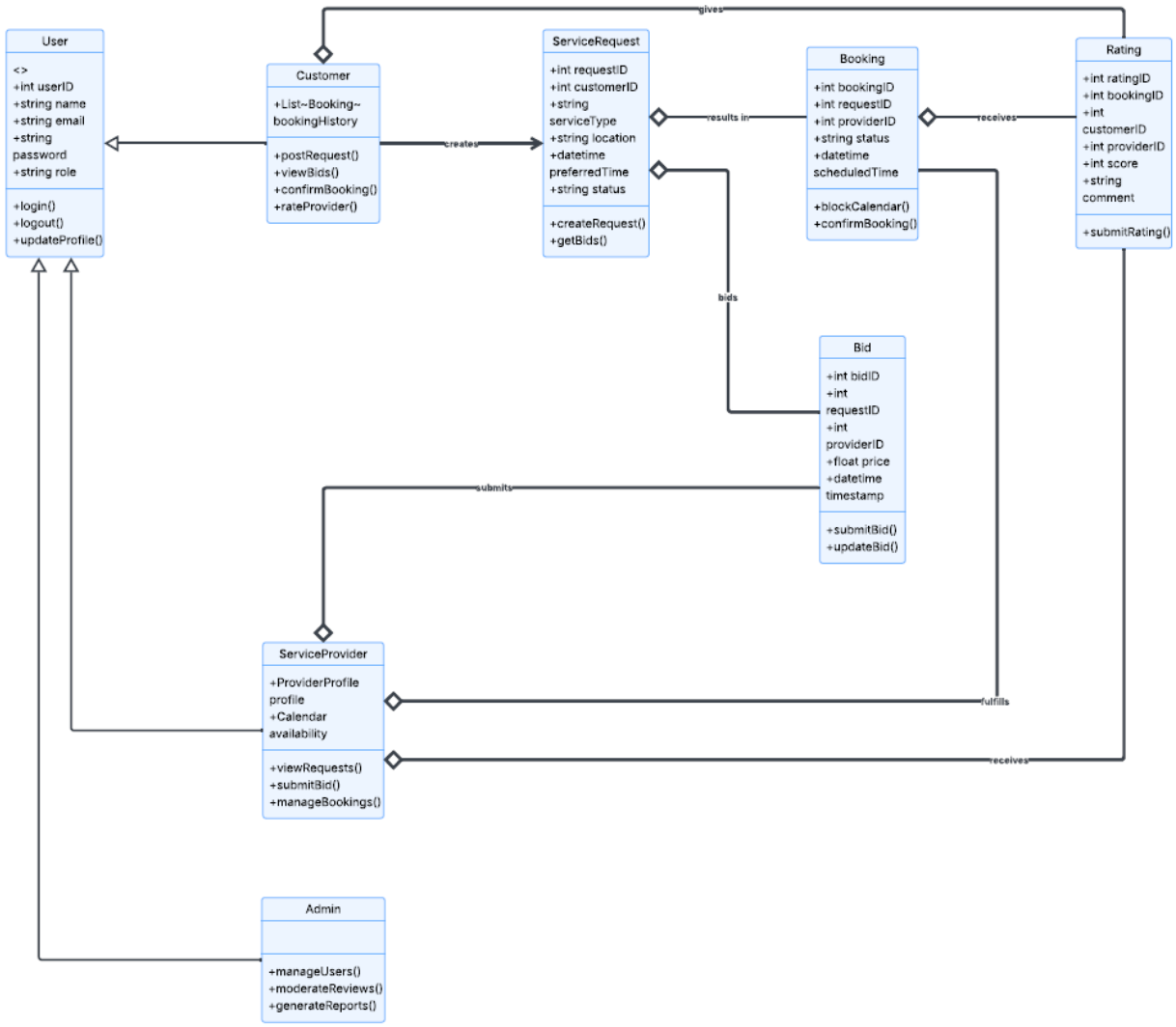

Figure 2: Booking Platform Class Diagram

The class diagram represents the main entities of the Booking Platform and their relationships, providing a blueprint of the system's structure. It shows an abstract User class with subclasses Customer, Service Provider, and Admin, each with attributes and methods relevant to their role. Customers can post service requests, view bids, confirm bookings, and rate providers, while Service Providers manage requests, submit bids, and track bookings. The ServiceRequest class captures details of each request, Bid represents offers from providers, Booking manages confirmed appointments with automated calendar updates, and Rating stores customer feedback. Relationships between classes illustrate how requests lead to bids, bids can result in bookings, and bookings may generate ratings, reflecting the end-to-end workflow of the platform. This diagram provides a clear visual of the system's structure, supporting both development and communication among stakeholders.

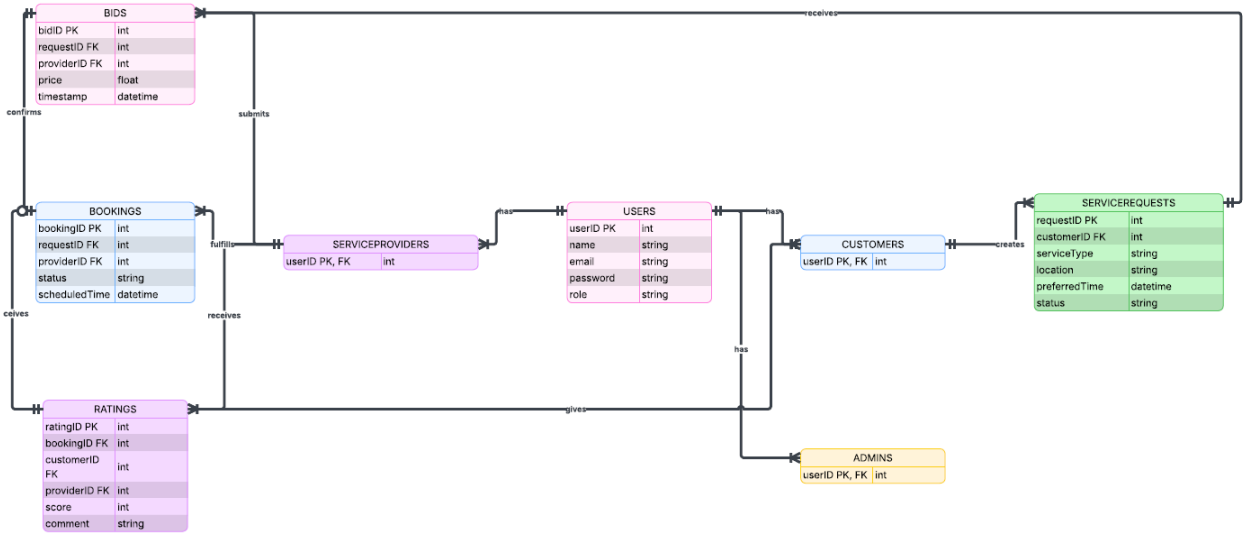

Figure 3: Booking Platform Entity Relationship Diagram

The Entity-Relationship Diagram (ERD) illustrates the database structure of the Booking Platform, showing how data is organized and connected to support the system's workflows. Core tables include Users, subdivided into Customers, Service Providers, and Admins, along with ServiceRequests, Bids, Bookings, and Ratings. Relationships depict that each Customer can create multiple Service Requests, each Request can receive multiple Bids from Providers, and confirmed Bids result in Bookings. Customers can optionally submit Ratings for completed Bookings, providing feedback to Providers. Primary keys uniquely identify records in each table, while foreign keys enforce relationships between entities, ensuring data integrity. This ERD provides a clear view of how information flows through the platform, supporting efficient data storage, retrieval, and management for all user interactions.

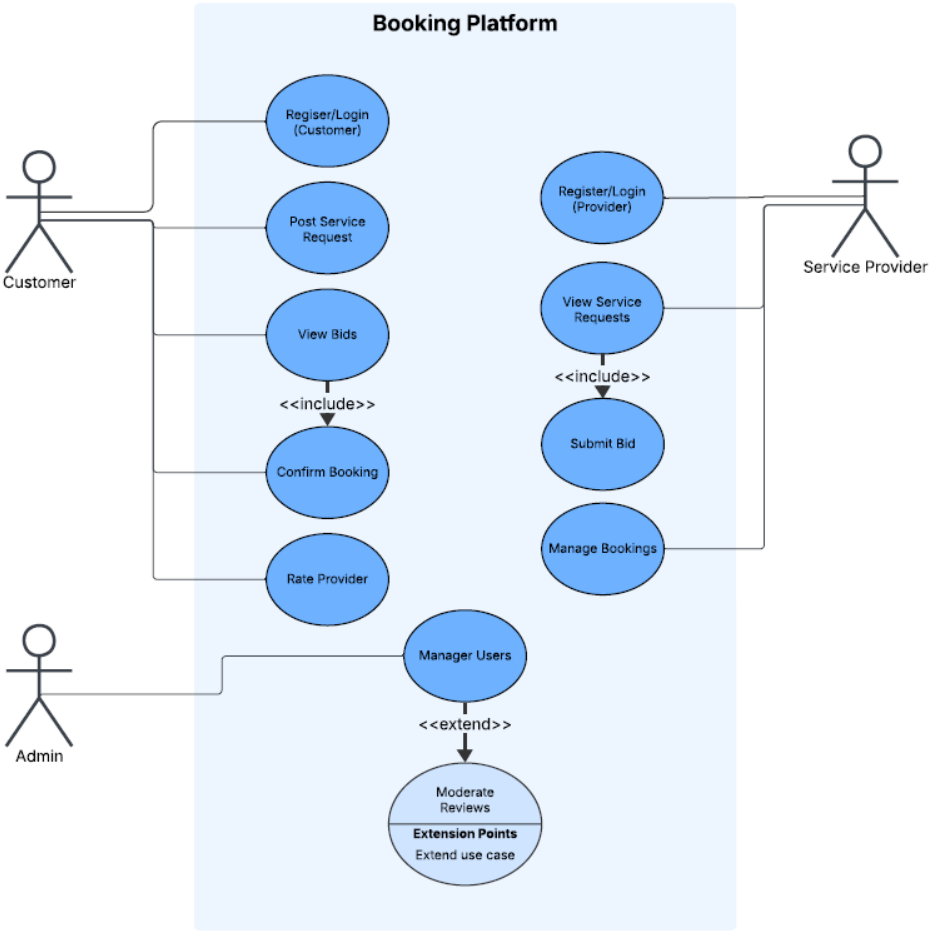

Figure 4: Booking Platform Use Case Diagram

The Use Case Diagram illustrates the interactions between the primary actors of the Booking Platform—Customer, Service Provider, and Admin—and the system's key functionalities. Customers can register or log in, post service requests, view bids, confirm bookings, and rate providers, with the "Confirm Booking" use case including the "View Bids" use case to show mandatory dependency. Service Providers register or log in, view service requests, submit bids, and manage bookings, with "Submit Bid" including "View Service Requests" as a required step. The Admin actor manages users and moderates reviews, with moderation optionally extending the user management process. Each actor is connected to their relevant use cases through associations

User Role Modelling

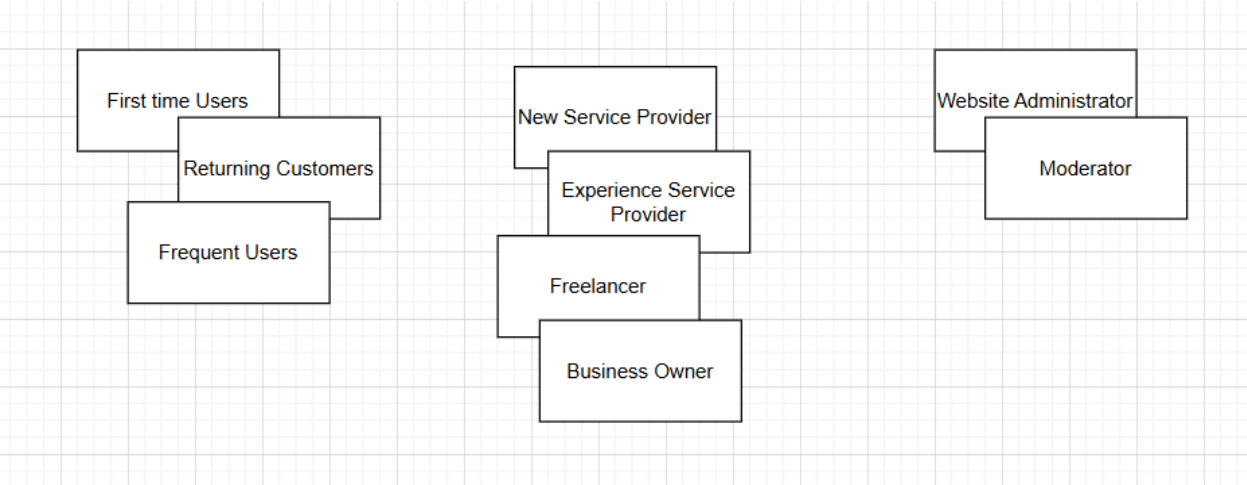

Figure 5. Organized User Role Cards

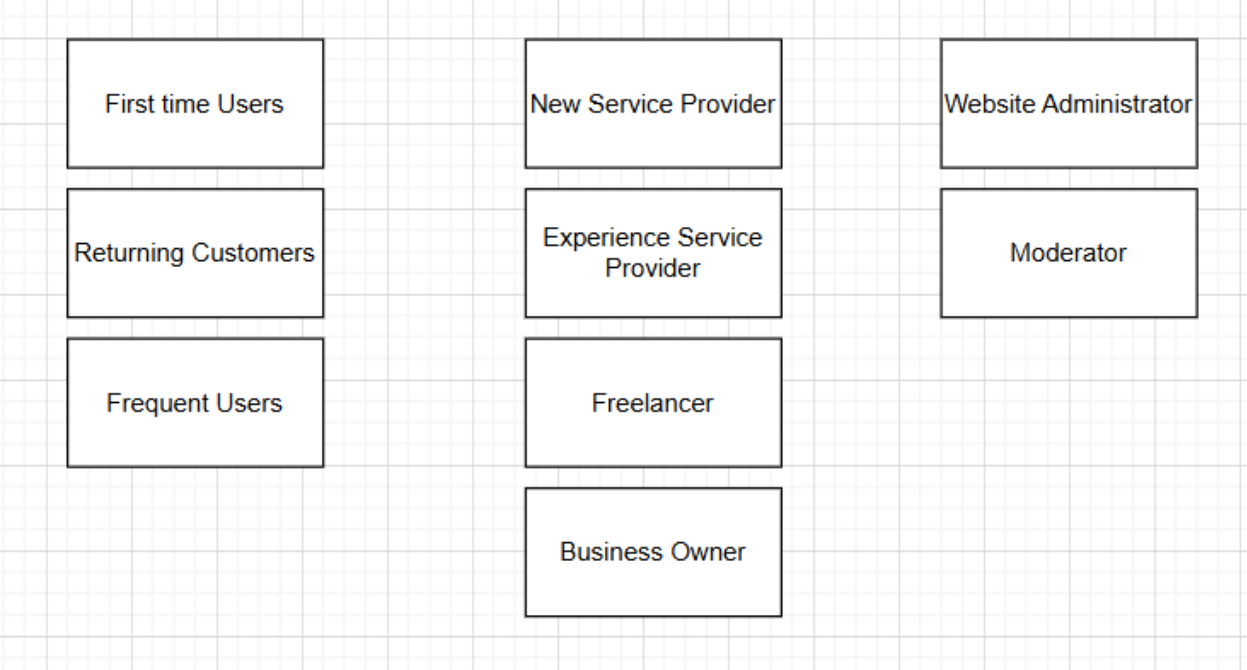

Figure 6. Consolidated User Role Cards

User Persona

Customer

- **Generic class**: All registered customers
- **Subclasses**: First-time customer, Returning customer, Frequent customer
- **Shared Goal**: Book local services conveniently and reliably

First-Time Customer

- Uses software occasionally (once every few weeks)
- Basic expertise in understanding local services
- Moderate proficiency with computers
- Beginner with the booking platform
- Goal: Complete first booking quickly and easily

Returning Customer

- Uses software occasionally to frequently (once a week or more)
- Moderate expertise in understanding local services
- Moderate proficiency with computers
- Intermediate proficiency with the booking platform
- Goal: Find trusted providers quickly and compare prices

Frequent Customer

- Uses software frequently (multiple times per week)
- High expertise in service requirements
- Above average computer proficiency
- Above average proficiency with the booking platform
- Goal: Efficiently manage multiple bookings and avoid scheduling conflicts

Service Provider

- **Generic class**: All providers offering services on the platform
- **Subclasses**: New Provider, Experienced Provider
- **Shared Goal**: Receive service requests, manage bookings, and maximize client opportunities

New Provider

- Uses software occasionally to frequently (few bookings per week)
- Beginner to moderate expertise in their service domain
- Moderate computer proficiency
- Beginner to intermediate proficiency with the booking platform
- Goal: Learn the platform and attract first customers

Experienced Provider

- Uses software frequently (daily or several times per week)
- High expertise in their service domain
- Moderate to high computer proficiency
- Intermediate to advanced proficiency with the booking platform
- Goal: Manage bookings efficiently, avoid conflicts, and maintain high ratings

Freelancer

- Uses software moderately
- Moderate expertise in the service domain
- Moderate to high computer proficiency
- Intermediate to advanced proficiency with the booking platform
- Goal: Attract customers in various fields

Business Owner

- Uses software occasionally to frequently (few bookings per week)
- Beginner to moderate expertise in their service domain
- Moderate computer proficiency
- Beginner to intermediate proficiency with the booking platform
- Goal: Check booking histories and services provided to make business decisions

Website Administrator

- **Generic class**: Platform management and oversight
- **Subclasses**: Moderator, System Administrator
- **Shared Goal**: Maintain platform reliability, fairness, and accurate data

Moderator

- Uses software daily as part of their job
- Above average expertise in platform operations
- Proficient with computers and software
- Intermediate to advanced proficiency with the booking platform
- Goal: Verify reviews, moderate content, and ensure fair platform usage

Low-Fidelity Prototype for 1st and 2nd Release

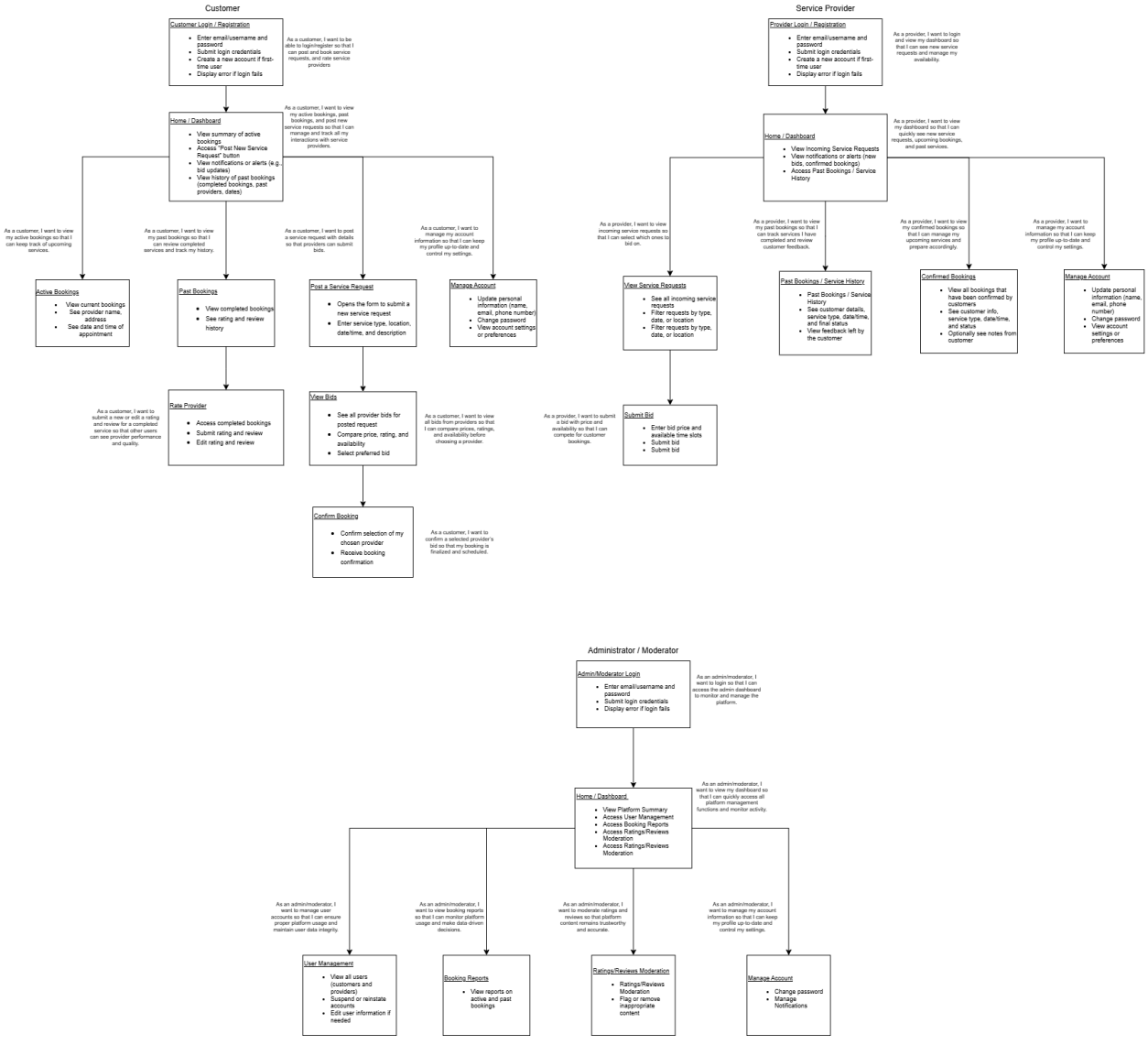

Figure 7. Consolidated Low Fidelity Prototype

The low-fidelity prototype for the booking platform illustrates the primary interactions and workflows for all user roles: customers, service providers, and admins/moderators. The prototype uses story boxes to represent key pages and functionalities for the 1st and 2nd releases, such as login, dashboards, service requests, confirmed and past bookings, submitting bids, moderating reviews, and managing account information. Each story box includes the main actions a user can perform along with a corresponding user story that captures the goal of that interaction, providing a clear view of how the system supports its users.

The prototype emphasizes the logical flow of the application, with dashboards acting as central hubs from which users can branch to specific features relevant to their role. For the 1st release, customers can post service requests, view bids, confirm bookings, and rate providers, while providers can view requests, submit bids, and monitor confirmed bookings. The 2nd release adds functionality such as viewing past bookings and managing account information for all users. Admins and moderators can access dashboards to manage users, view booking reports, moderate ratings and reviews, and maintain their account settings.

Overall, this low-fidelity prototype provides a structured, visual representation of the system's functionality for both releases, supports discussion and feedback during workshops, and ensures that the key workflows for each user role are captured and validated prior to development.

Estimating Stories for 1st and 2nd Release

Estimation Method

The team used the Wide-Band Delphi approach, where each member estimates story points independently (blind estimation) before discussing differences and agreeing on a final estimate. This reduces bias and groupthink and ensures all developers contribute to the estimates.

Methodologies

Estimation scale used: 1, 3, 5, 8 (increasing complexity/effort).

Release 1 Story Estimates

Story / Screen ID	Story Summary	Estimated Points
SS-R1-1	Login / Register	5
SS-R1-2	Create service request	5
SS-R1-3	View bids & compare	3
SS-R1-4	Confirm booking	8
SS-R1-1	Provider profile setup	5

SS-R1-2	View open requests	3
SS-R1-3	Booking confirmation + calendar block	8
AD-R1-1	Admin basic dashboard	3
AD-R1-2	User management	3
AD-R1-3	Ratings/Review Moderation	3

Release 1 Total: **46 points**

Release 2 Story Estimates

Story / Screen ID	Story Summary	Estimated Points
SS-R2-1	Booking status/history	3
SS-R2-2	Rate & review provider	3
SS-R2-3	Manage account (customer)	3
SP-R2-1	Manage availability	5
SP-R2-2	Past bookings/history	3
SP-R2-3	Manage account (provider)	3
AD-R2-1	Moderate reviews	3
AD-R2-2	Basic reporting	5
AD-R2-3	Manage account (admin)	3

Release 2 Total: 31 points

The team assigned story points to each user story for the 1st and 2nd releases based on relative complexity and effort. The 1st release, totaling 46 story points, covers core functionality required for the system to operate, including login and registration, dashboards, posting service requests, viewing and comparing bids, confirming bookings, provider profile setup, booking confirmation with calendar blocking, and basic admin and moderation tasks. These stories represent the foundational workflows for customers, service providers, and administrators.

The 2nd release, totaling 31 story points, focuses on enhancements and secondary features, such as viewing booking history, rating and reviewing providers, managing accounts for all user roles, managing provider availability, and basic reporting. These stories improve usability, allow

users to track past activities, and provide administrative oversight. The distribution of points across releases ensures that development effort is balanced and that each release delivers a functional and testable set of features.

Overall, the story point estimates provide a structured basis for sprint planning, resource allocation, and tracking progress, while maintaining consistency and realistic expectations across both releases.

Story Prioritization for 1st and 2nd Release

Story prioritization was conducted using the MoSCoW method, which classifies user stories into Must Have, Should Have, Could Have, and Won't Have categories. This approach helps ensure that core system functionality is delivered first while allowing less critical features to be implemented in later releases. Prioritization decisions were based on business value, system dependency, frequency of use, and impact on the overall booking workflow.

Must Have (Release 1 – Core Features)

The following stories represent essential system functionality and must be implemented for the platform to operate:

- User registration and login
- Service request creation
- Provider profile setup
- Viewing open service requests
- Submitting provider bids
- Booking confirmation
- Automatic calendar blocking to prevent double bookings
- Admin dashboard and user management

These stories form the core end-to-end workflow (request → bid → confirm → schedule) and are required to demonstrate a working booking platform.

Should Have (Release 2 – Important Enhancements)

These stories enhance usability and trust but are not required for basic system operation:

- Viewing booking status and history
- Rating and reviewing service providers
- Account management
- Provider availability management
- Admin reporting and review moderation

These features improve user experience and service quality but depend on the successful implementation of the core booking workflow.

Could Have:

- Advanced bid filtering

- Notification preferences

Won’t Have:

- Online payments

- Third-party calendar integration

Prioritization Rationale

Core booking features were prioritized as Must Have to ensure the platform can demonstrate its primary value: preventing scheduling conflicts and enabling transparent service booking. Lower-priority stories were scheduled for later releases to support system flow without delaying critical functionality. Supporting documentation for prioritization decisions reflects a balance between technical feasibility, business value, and project constraints.